

프로그래밍응용개념

#02 변수

국립한밭대학교 인공지능 소프트웨어학과

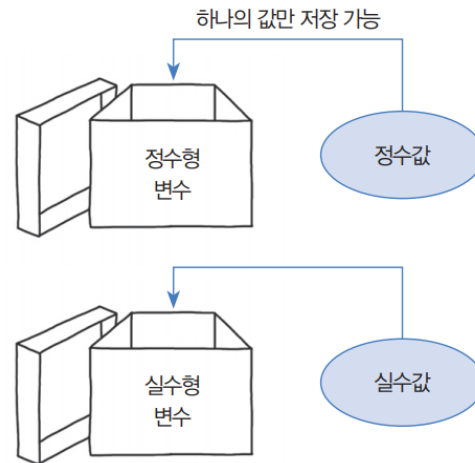
이성민

변수

- 정수, 실수, 문자, 불리언 자료형
- 문자열 메소드
- 타입 추론

변수

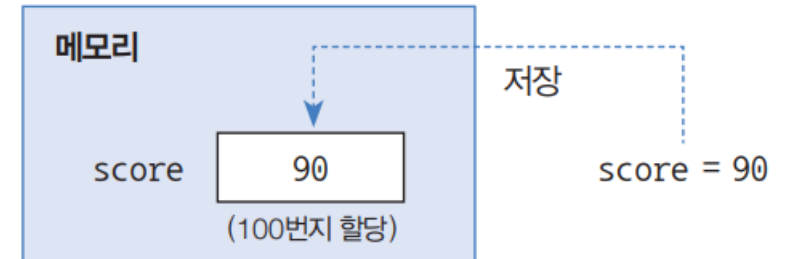
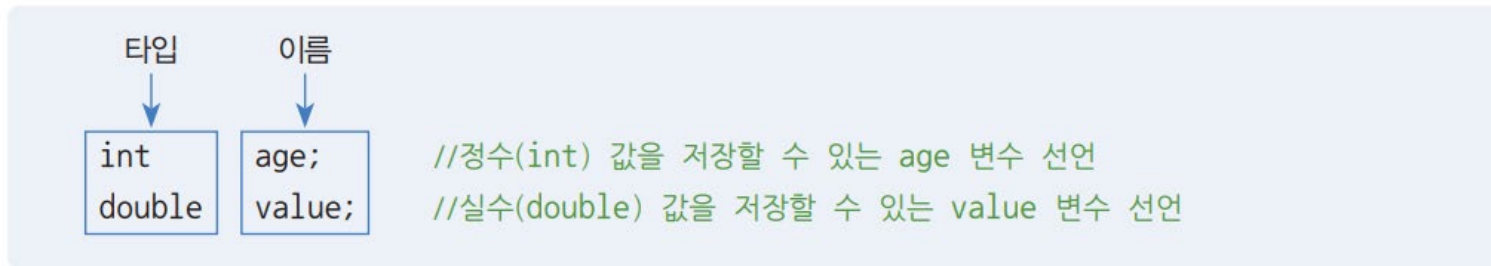
- 변수 (variable):
 - 하나의 값을 저장할 수 있는 메모리 주소에 붙여진 이름
 - 하나의 변수에는 하나의 타입(정수형, 실수형 등) 만 저장 가능



변수

- 변수 선언

- 변수를 사용하려고 하면 변수 선언이 필수
- 어떤 타입의 데이터를 저장할 것인지, 변수 이름이 무엇인지 결정
- 변수에 최초로 값이 대입될 때 메모리에 할당되고, 해당 메모리에 값 저장



변수

- 변수의 이름
 - Camel 스타일 (여러 단어를 혼합하는 경우 대문자로 띄어쓰기 구분)
- 변수명은 소문자로 시작
 - score
 - mathScore
 - sportscar
- 소스 파일명(클래스명)은 대문자로 시작
 - Week.java
 - MemberGrade.java
 - ProductKind.java

변수

- 변수 초기화
 - 변수에 최초로 값을 넣는 행위
 - 변수가 사용되기 이전에 초기화 되어 있어야 함

```
//변수 value 선언
int value;

//연산 결과를 변수 result의 초기값으로 대입
int result = value + 10;

//변수 result 값을 읽고 콘솔에 출력
System.out.println(result);
```

변수값 출력

- `println()` 메소드로 변수값 출력
 - 모니터에 값을 출력하기 위해 `System.out.println()` 이용
 - 출력 방법에 따라 `println()`, `print()`, `printf()`를 사용할 수 있음

메소드	의미
<code>println(내용);</code>	괄호 안의 내용을 출력하고 행을 바꿔라.
<code>print(내용);</code>	괄호 안의 내용을 출력하고 행은 바꾸지 말아라.
<code>printf("형식문자열", 값1, 값2, ...);</code>	형식 문자열에 맞추어 뒤의 값을 출력해라.

정수 타입 (정수 자료형)

- 정수 타입 종류

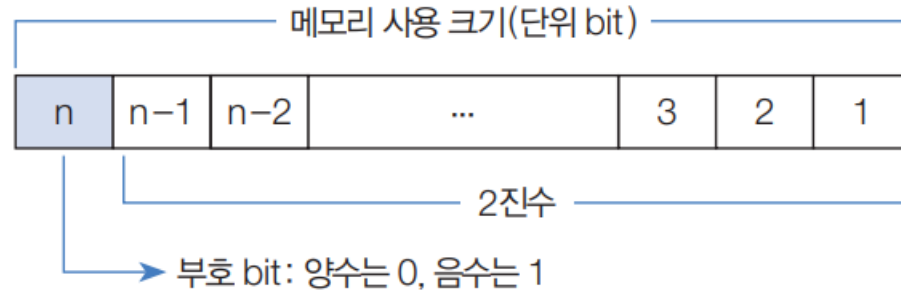
- 변수 타입에 따라 메모리에 할당되는 크기와 저장되는 값의 범위가 다름

타입	메모리 크기		저장되는 값의 허용 범위	
byte	1byte*	8bit	$-2^7 \sim (2^7-1)$	-128 ~ 127
short	2byte	16bit	$-2^{15} \sim (2^{15}-1)$	-32,768 ~ 32,767
char	2byte	16bit	$0 \sim (2^{16}-1)$	0 ~ 65535 (유니코드)
int	4byte	32bit	$-2^{31} \sim (2^{31}-1)$	-2,147,483,648 ~ 2,147,483,647
long	8byte	64bit	$-2^{63} \sim (2^{63}-1)$	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807

* 1byte = 8bit, bit는 0과 1이 저장되는 단위

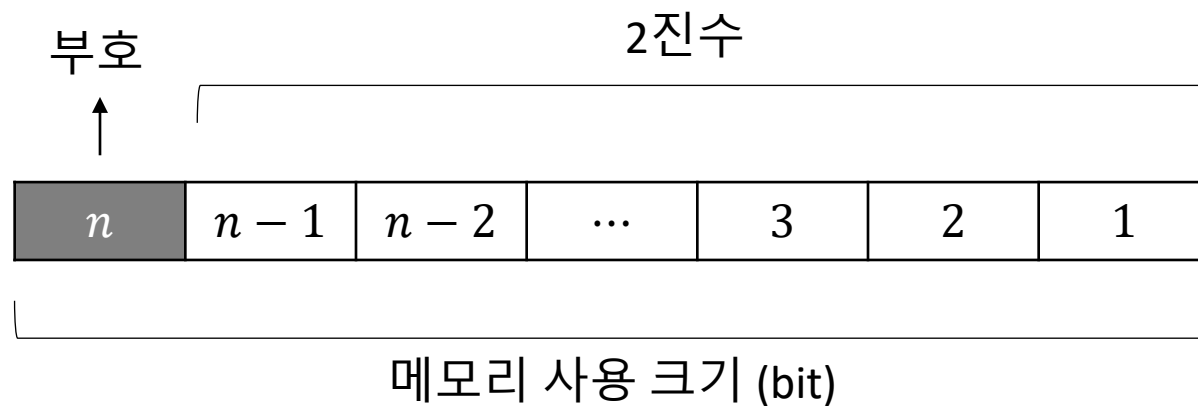
정수 타입 (정수 자료형)

- 정수 타입 종류
 - 메모리 크기를 n 이라고 했을 때, 2진수로 저장



정수 타입 (정수 자료형)

- 음수 표현



0	$n-1$	$n-2$	...	3	2	1
---	-------	-------	-----	---	---	---

 → 값의 범위: $0 \sim 2^{n-1} - 1$

1	$n-1$	$n-2$	...	3	2	1
---	-------	-------	-----	---	---	---

 → 값의 범위: $-2^{n-1} \sim -1$

정수 타입 (정수 자료형)

- 음수 표현

0	1	1	1	1	1	1	1	→	127
0	1	1	1	1	1	1	0	→	126
0	0	0	0	0	0	0	1	→	1
0	0	0	0	0	0	0	0	→	0
1	1	1	1	1	1	1	1	→	-1
1	0	0	0	0	0	0	1	→	-127
1	0	0	0	0	0	0	0	→	-128

정수 타입 (정수 자료형)

- byte 타입 (1 byte)

```
byte var1 = -128;  
byte var2 = -30;  
byte var3 = 0;  
byte var4 = 30;  
byte var5 = 127;  
//byte var6 = 128; //결과일 예러(Type mismatch: cannot covert from int to byte)  
  
System.out.println(var1);  
System.out.println(var2);  
System.out.println(var3);  
System.out.println(var4);  
System.out.println(var5);
```

- 리터럴 (literal)
 - 프로그래머가 직접 입력한 값

정수 타입 (정수 자료형)

- short 타입 (2 byte)

```
short var1 = 10;  
short var2 = 4;  
// short var3 = 32768; // 컴파일 에러(Type mismatch: cannot covert from int to short)  
  
System.out.println(var1);  
System.out.println(var2);
```

정수 타입 (정수 자료형)

- int 타입 (4 byte)
 - 2진수: 0B 혹은 0b로 시작
 - 8진수: 0으로 시작
 - 10진수: 일반적인 숫자 작성
 - 16진수: 0X 혹은 0x로 시작
 - A=10, B=11, C=12, D=13, E=14, F=15
 - a=10, b=11, c=12, d=13, e=14, f=15

```
int var1 = 0b1011;    //2진수
int var2 = 0206;       //8진수
int var3 = 365;        //10진수
int var4 = 0xB3;       //16진수

System.out.println("var1: " + var1);
System.out.println("var2: " + var2);
System.out.println("var3: " + var3);
System.out.println("var4: " + var4);
```

정수 타입 (정수 자료형)

- long 타입 (8 byte)
 - 정수타입 리터럴은 자동으로 int형으로 인식됨
 - int 범위를 벗어나는 리터럴을 만들고 싶으면 맨 뒤에 L 혹은 l을 추가

```
long var1 = 10;  
long var2 = 20L;  
//long var3 = 10000000000000; //컴파일러는 int로 간주하기 때문에 에러 발생시킬  
long var4 = 10000000000000L;  
  
System.out.println(var1);  
System.out.println(var2);  
System.out.println(var4);
```

문자 타입 (문자 자료형)

- 문자 리터럴과 char
 - 문자 리터럴: 하나의 문자를 작은 따옴표(')로 감싼 것
 - 문자 리터럴을 유니코드로 저장할 수 있도록 char 타입 제공

```
char var1 = 'A';    //'A' 문자와 매핑되는 숫자: 65로 대입  
char var3 = '가';   //'가' 문자와 매핑되는 숫자: 44032로 대입
```

```
char c = 65;        //10진수 65와 매핑되는 문자: 'A'  
char c = 0x0041;    //16진수 0x0041과 매핑되는 문자: 'A'
```

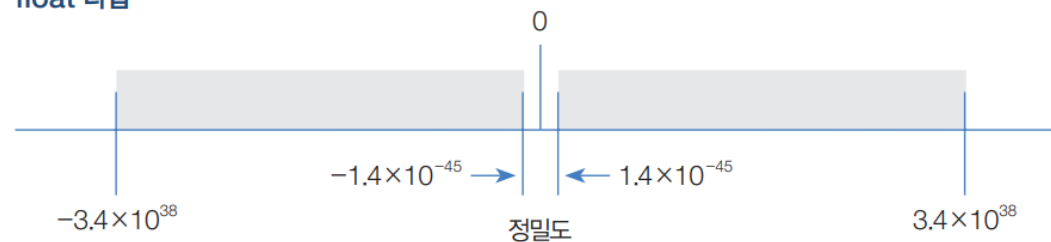

실수 타입 (실수 자료형)

- float 과 double 타입

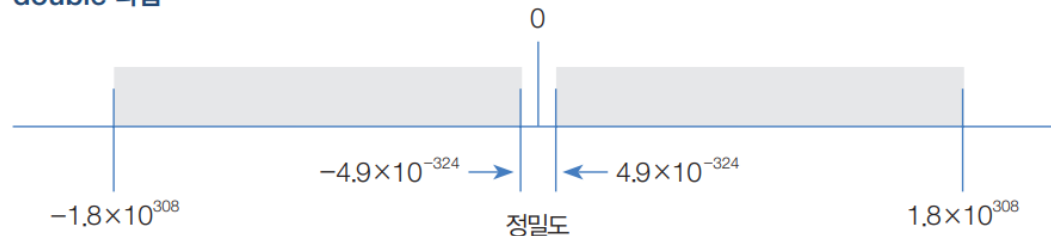
- double 타입이 float 타입보다 큰 실수를 저장할 수 있고 정밀도도 높음

타입	메모리 크기		저장되는 값의 허용 범위(양수 기준)	유효 소수 이하 자리
float	4 byte	32 bit	$1.4 \times 10^{-45} \sim 3.4 \times 10^{38}$	7자리
double	8 byte	64 bit	$4.9 \times 10^{-324} \sim 1.8 \times 10^{308}$	15자리

float 타입



double 타입

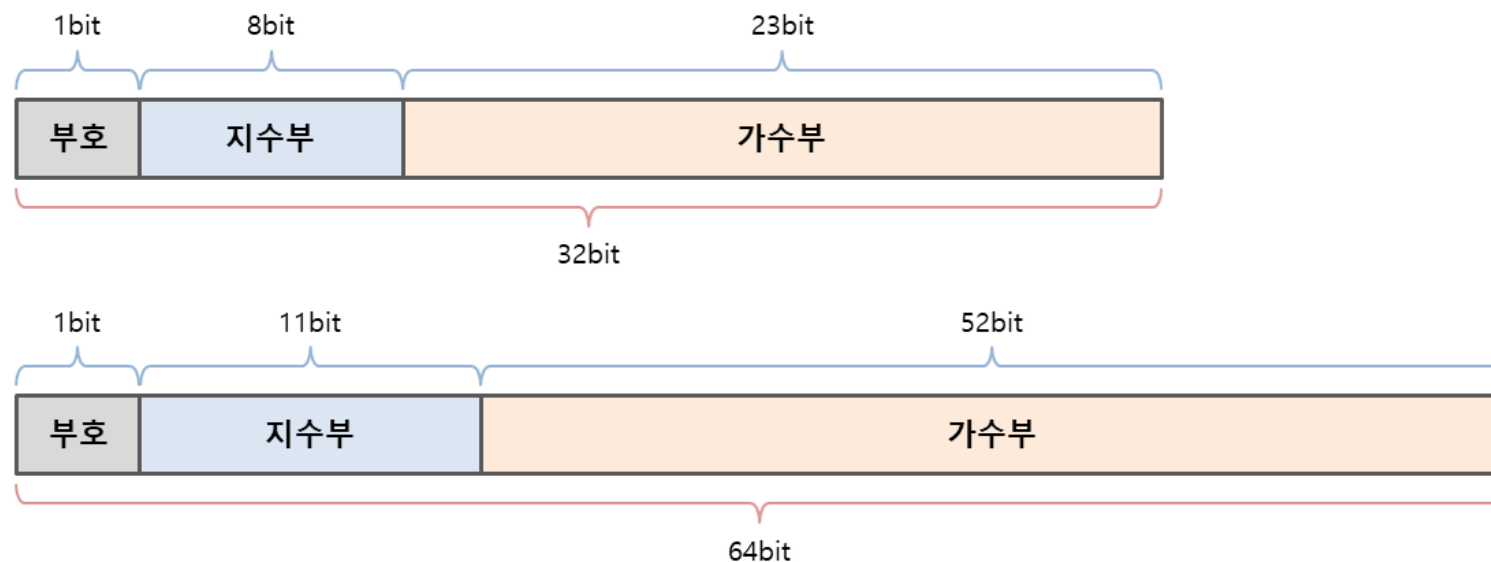


실수 타입 (실수 자료형)

- float 과 double 타입

- double 타입이 float 타입보다 큰 실수를 저장할 수 있고 정밀도도 높음
- 부동소수점 방식

$$\begin{array}{c} \pm \\ \uparrow \\ \text{부호} \end{array} \begin{array}{c} m \\ \uparrow \\ \text{가수} \end{array} \times \begin{array}{c} n \\ \uparrow \\ \text{기수} \end{array} \begin{array}{c} p \\ \downarrow \\ \text{지수} \end{array}$$



실수 타입 (실수 자료형)

- float 과 double 타입

- double 타입이 float 타입보다 큰 실수를 저장할 수 있고 정밀도도 높음

- 예) 263.3 을 부동소수점 방식으로 표현

- $263.3 = (\text{2진수}) 100000111.010011001100110\dots$
 $= (\text{2진수}) 1.00000111010011001100110\dots \times 2^8$

- 부호비트 (1 bit) : 0 (양수)
 - 지수비트 (8 bit) : 10000111 ($127 + 8 = 135$)
 - 여기서 127은 bias로 부호를 간편하게 표현하기 위함
 - 가수비트 (23 bit) : 00000111010011001100110

실제 지수	저장된 값
-127	0
-126	1
...	...
0	127
1	128
...	...
128	255

실수 타입 (실수 자료형)

- float 과 double 타입
 - 실수타입 리터럴은 자동으로 double형으로 인식됨
 - float 범위를 벗어나는 리터럴을 만들고 싶으면 맨 뒤에 F 혹은 f를 추가

```
//정밀도 확인  
float var1 = 0.1234567890123456789f;  
double var2 = 0.1234567890123456789;  
System.out.println("var1: " + var1);  
System.out.println("var2: " + var2);
```

실수 타입 (실수 자료형)

- float 과 double 타입
 - e혹은 E가 포함된 리터럴: $5e2 = 5 \times 100$

논리 타입 (논리 자료형)

- boolean 타입
 - true 혹은 false로 구성됨

```
boolean stop = true;  
boolean stop = false;
```

- 주로 두 가지 상태값을 저장하는 경우에 사용

	연산식	
int x = 10;		
boolean result =	(x == 20);	//변수 x의 값이 20인가?
boolean result =	(x != 20);	//변수 x의 값이 20이 아닌가?
boolean result =	(x > 20);	//변수 x의 값이 20보다 큰가?
boolean result =	(0 < x && x < 20);	//변수 x의 값이 0보다 크고, 20보다 적은가?
boolean result =	(x < 0 x > 200);	//변수 x의 값이 0보다 적거나 200보다 큰가?

문자열 타입 (문자열 자료형)

- String 타입

- 문자열 (String): 큰 따옴표(“”)로 감싼 문자들

```
String var1 = "A";  
String var2 = "홍길동";
```

- 이스케이프 문자: 문자열 내부에 역슬래시(\)가 붙은 특수 문자

이스케이프 문자	
\"	" 문자 포함
\'	' 문자 포함
\\	\ 문자 포함
\u16진수	16진수 유니코드에 해당하는 문자 포함
\t	출력 시 탭만큼 띄움
\n	출력 시 줄바꿈(라인피드)
\r	출력 시 캐리지 리턴

문자열 타입 (문자열 자료형)

- String 타입
 - 참고: println 메소드는 print + \n 과 같음

```
String name = "홍길동";  
String job = "프로그래머";  
System.out.println(name);  
System.out.println(job);  
  
String str = "나는 \"자바\"를 배웁니다..";  
System.out.println(str);  
  
str = "번호\t이름\t직업 ";  
System.out.println(str);  
  
System.out.print("나는\n");  
System.out.print("자바를\n");  
System.out.print("배웁니다.");
```

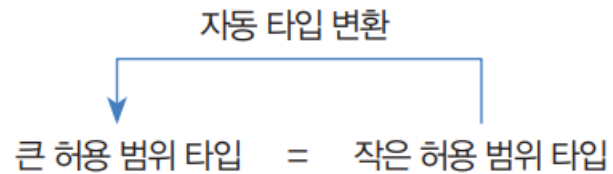


```
홍길동  
프로그래머  
나는 "자바"를 배웁니다..  
번호 이름 직업  
나는  
자바를  
배웁니다.
```

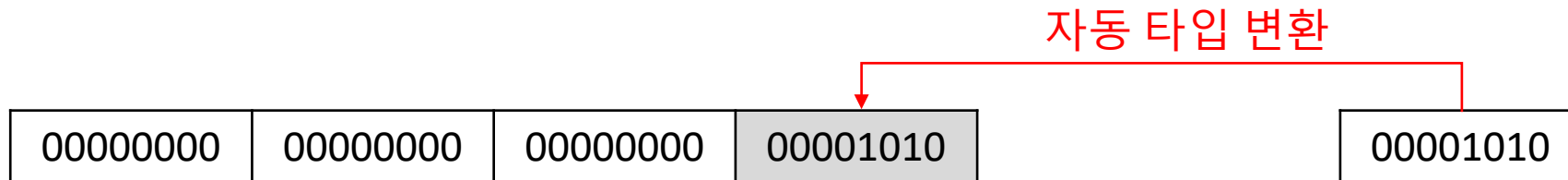

타입 변환 (형 변환)

- 자동 타입 변환

- 데이터 타입을 다른 타입으로 변환하는 것
- 값의 허용 범위가 작은 타입이 허용 범위가 큰 타입으로 대입될 때 발생



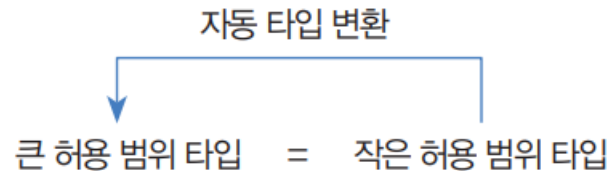
byte < short, char < int < long < float < double



타입 변환 (형 변환)

- 자동 타입 변환

- 정수 타입이 실수 타입으로 대입되면 무조건 자동 타입 변환이 됨
- 예외: char 타입보다 허용 범위가 작은 byte 타입은 char 타입으로 자동 변환될 수 없음



byte < short, char < int < long < float < double

타입 변환 (형 변환)

- 자동 타입 변환

```
// 자동 타입 변환
byte byteValue = 10;
int intValue = byteValue;
System.out.println("intValue: " + intValue);

char charValue = '가';
intValue = charValue;
System.out.println("가의 유니코드: " + intValue);

intValue = 50;
long longValue = intValue;
System.out.println("longValue: " + longValue);

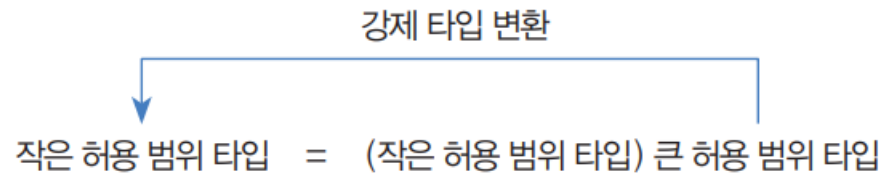
longValue = 100;
float floatValue = longValue;
System.out.println("floatValue: " + floatValue);

floatValue = 100.5F;
double doubleValue = floatValue;
System.out.println("doubleValue: " + doubleValue);
```

타입 변환 (형 변환)

- 강제 형 변환

- 큰 허용 범위 타입을 작은 허용 범위 타입으로 쪼개어서 저장하는 것
- 캐스팅 연산자로 괄호()를 사용하며, 괄호 안에 들어가는 타입은 쪼개는 단위

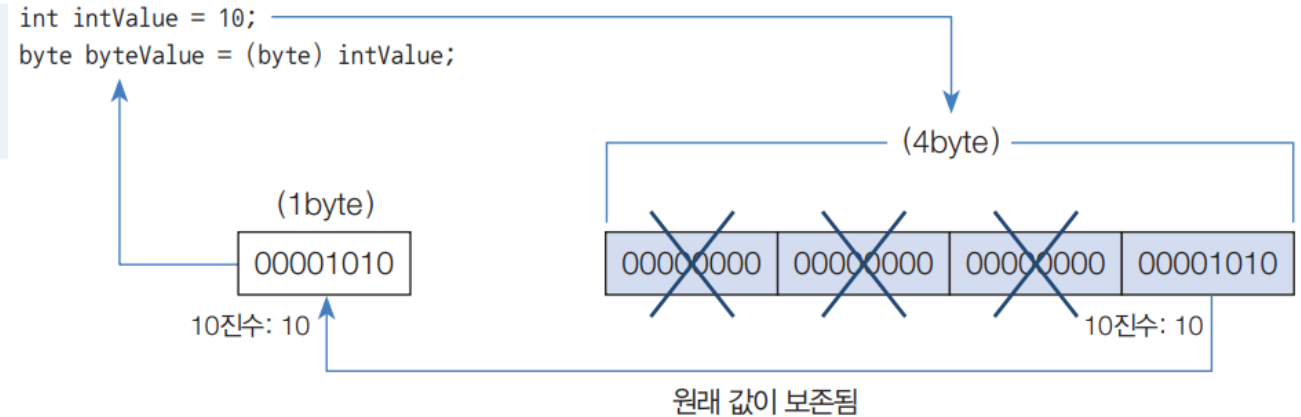


- long → int
- int → char
- 실수 → 정수

타입 변환 (형 변환)

- 강제 형 변환
 - 예: `int` → `byte` 강제 타입 변환

```
int intValue = 10;  
byte byteValue = (byte) intValue; //강제 타입 변환
```



타입 변환 (형 변환)

- 강제 형 변환

```
int var1 = 10;
byte var2 = (byte) var1;
System.out.println(var2);    //강제 타입 변환 후에 10이 그대로 유지

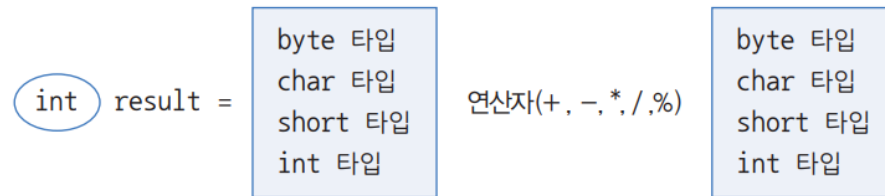
long var3 = 300;
int var4 = (int) var3;
System.out.println(var4);    //강제 타입 변환 후에 300이 그대로 유지

int var5 = 65;
char var6 = (char) var5;
System.out.println(var6);    //'A'가 출력

double var7 = 3.14;
int var8 = (int) var7;
System.out.println(var8);    //3이 출력
```

타입 변환 (형 변환)

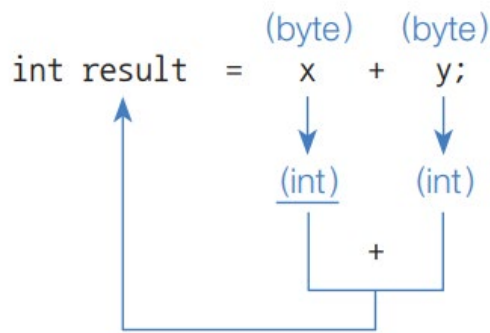
- 연산식에서 자동 타입 변환
 - 정수 타입 변수가 산술 연산식에서 피연산자로 사용되면 int 타입보다 작은 byte, short 타입 변수는 int 타입으로 자동 변환되어 연산 수행



byte 타입 변수가 피연산자로 사용된 경우	int 타입 변수가 피연산자로 사용된 경우
<pre>byte x = 10; byte y = 20; byte result = x + y; //컴파일 에러 int result = x + y;</pre>	<pre>int x = 10; int y = 20; int result = x + y;</pre>

타입 변환 (형 변환)

- 연산식에서 자동 타입 변환
 - 정수 타입 변수가 산술 연산식에서 피연산자로 사용되면 int 타입보다 작은 byte, short, char 타입 변수는 int 타입으로 자동 변환되어 연산 수행
 - byte 변수가 피연산자로 사용되면 변수값은 int 값으로 연산되며, 결과값 역시 byte 변수가 아닌 int 변수에 저장해야 함 (short, char 도 마찬가지)



```
char v6 = 'A';
char v7 = 1;
int result4 = v6 + v7; //int 타입으로 변환후 연산
System.out.println("result4: " + result4);
System.out.println("result4: " + (char)result4);
```


타입 변환 (형 변환)

- 연산식에서 자동 타입 변환
 - long 변수가 피연산자로 사용되면 변수값은 long 값으로 연산되며, 결과값 역시 long 변수에 저장해야 함

```
byte v3 = 10;  
int v4 = 100;  
long v5 = 1000L;  
long result3 = v3 + v4 + v5;    //long 타입으로 변환후 연산  
System.out.println("result3: " + result3);
```

타입 변환 (형 변환)

- 연산식에서 자동 타입 변환
 - double 변수가 피연산자로 사용되면 변수값은 double 값으로 연산되며, 결과값 역시 double 변수에 저장해야 함

```
int v9 = 10;  
double result6 = v9 / 4.0; //double 타입으로 변환후 연산  
System.out.println("result6: " + result6);
```

```
float v12 = 1;  
double v13 = 2;  
double result8 = v12 / v13; //double 타입이 연산에 포함되면 double 타입으로 변환후 연산  
System.out.println("result7: " + result8);
```

타입 변환 (형 변환)

- 문자열을 기본 타입으로 변환

변환 타입	사용 예
String → byte	<code>String str = "10"; byte value = Byte.parseByte(str);</code>
String → short	<code>String str = "200"; short value = Short.parseShort(str);</code>
String → int	<code>String str = "3000000"; int value = Integer.parseInt(str);</code>
String → long	<code>String str = "4000000000000"; long value = Long.parseLong(str);</code>
String → float	<code>String str = "12.345"; float value = Float.parseFloat(str);</code>
String → double	<code>String str = "12.345"; double value = Double.parseDouble(str);</code>
String → boolean	<code>String str = "true"; boolean value = Boolean.parseBoolean(str);</code>

타입 변환 (형 변환)

- 문자열을 기본 타입으로 변환
 - 기본 타입의 값을 문자열로 변경할 때는 String.valueOf() 메소드 사용

```
int value1 = Integer.parseInt(s: "10");
double value2 = Double.parseDouble(s: "3.14");
boolean value3 = Boolean.parseBoolean(s: "true");

System.out.println("value1: " + value1);
System.out.println("value2: " + value2);
System.out.println("value3: " + value3);

String str1 = String.valueOf(i: 10);
String str2 = String.valueOf(d: 3.14);
String str3 = String.valueOf(b: true);

System.out.println("str1: " + str1);
System.out.println("str2: " + str2);
System.out.println("str3: " + str3);
```

타입 변환 (형 변환)

- 문자열 결합 (+ 연산자)
 - 피연산자가 모두 숫자 → 덧셈 연산
 - 피연산자 중 문자열이 있을 때 → 문자열 자동 변환 후 결합

```
//숫자 연산
int result1 = 10 + 2 + 8;
System.out.println("result1: " + result1);

//결합 연산
String result2 = 10 + 2 + "8";
System.out.println("result2: " + result2);

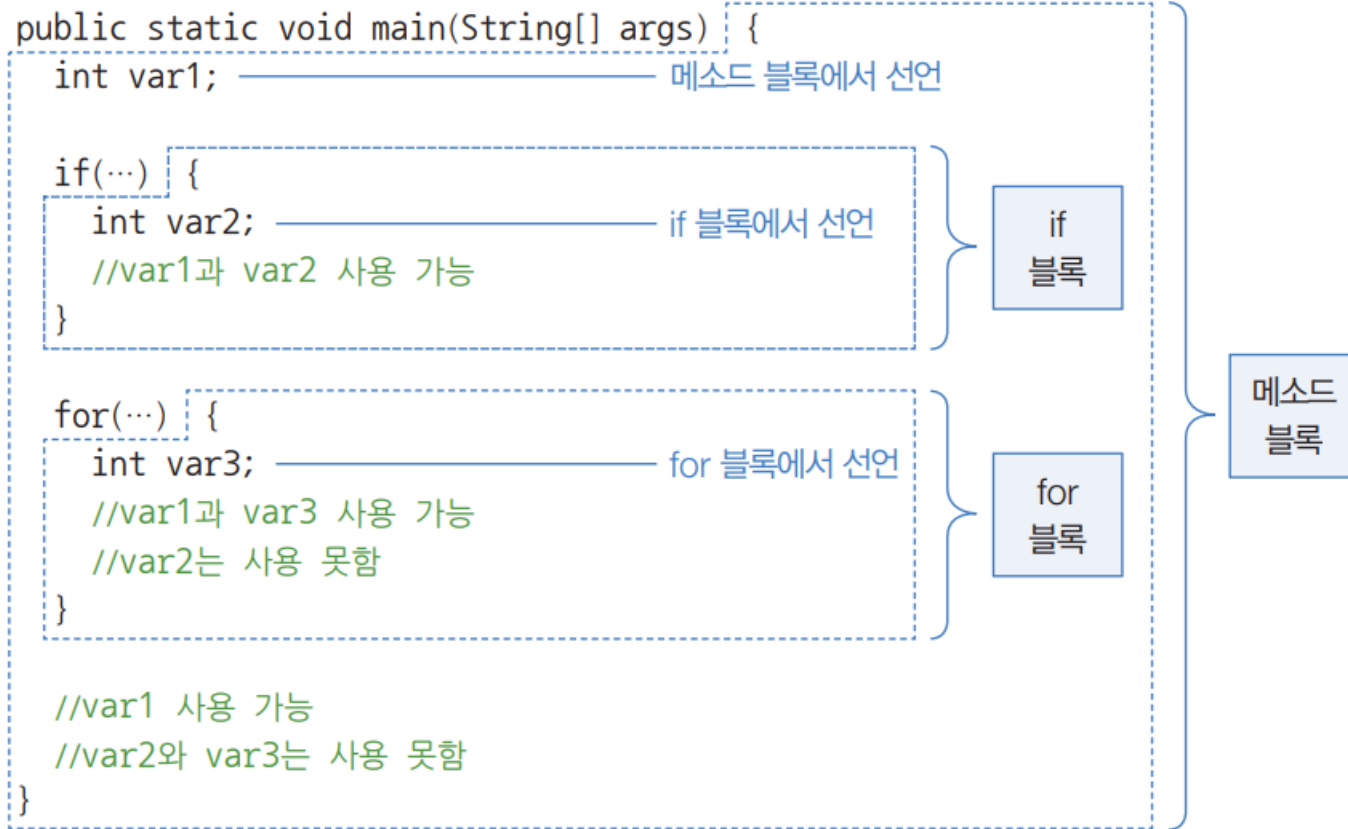
String result3 = 10 + "2" + 8;
System.out.println("result3: " + result3);

String result4 = "10" + 2 + 8;
System.out.println("result4: " + result4);

String result5 = "10" + (2 + 8);
System.out.println("result5: " + result5);
```

변수 사용 범위

- 변수 범위를 나타내는 중괄호 {} 블록



변수값 출력

- printf() 메소드로 변수값 출력
 - printf()의 형식 문자열에서는 %와 conversation(변환 문자)를 필수로 작성하고 나머지는 생략 가능

```
int value = 123;
System.out.printf("상품의 가격:%d원\n", value);
System.out.printf("상품의 가격:%6d원\n", value);
System.out.printf("상품의 가격:%-6d원\n", value);
System.out.printf("상품의 가격:%06d원\n", value);

double area = 3.14159 * 10 * 10;
System.out.printf("반지름이 %d인 원의 넓이:%10.2f\n", 10, area);

String name = "홍길동";
String job = "도적";
System.out.printf("%6d | %-10s | %10s\n", 1, name, job);
```

변수값 출력

- 형식화된 문자열

형식화된 문자열		설명	출력 형태
정수	%d	정수	123
	%6d	정수 6자리, 왼쪽 공백	___123
	%-6d	정수 6자리, 오른쪽 공백	123___
	%06d	정수 6자리, 왼쪽 공백 0 채움	000123
실수	%f	정수 + 소수점 + 소수 6자리, 왼쪽 공백 0 채움	123.450000
	%10.2f	정수 7자리 + 소수점 + 소수 2자리, 왼쪽 공백	____123.45
	%-10.2f	정수 7자리 + 소수점 + 소수 2자리, 오른쪽 공백	123.45_____
	%010.2f	정수 7자리 + 소수점 + 소수 2자리, 왼쪽 공백 0 채움	000123.45
문자열	%s	문자열	abc
	%6s	문자열 6자리, 왼쪽 빈자리 공백	___abc
	%-6s	문자열 6자리, 오른쪽 빈자리 공백	abc___

변수 타입 추론

- var 타입
 - 리터럴의 타입을 보고 컴파일러가 변수의 타입을 추론해서 저장함
 - 따라서, 반드시 초기화 해줘야 함
 - 한번 추론된 타입은 변경되지 않음

```
var intNum = 1;
var doubleNum = 3.14;
var charLet = 'A';
var StringWord = "안녕하세요";

// 아래와 같은 사용 불가
// var notInit; // 컴파일러가 타입을 추론할 수 없는 상황
// intNum = 2.0; // 다른 타입으로 변경하는 경우
```

